Outlier

Eunomia

Attempter Intro

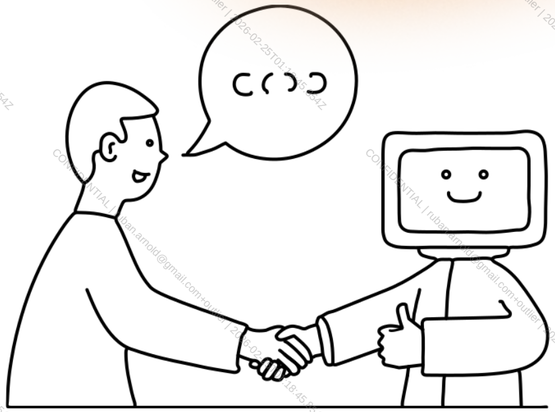
CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

Welcome!

The goal of this project is to improve the ability of AI models to act as **effective agents** assisting in practical, everyday **coding tasks.**

In these tasks you will:

- Read **realistic, multi-turn coding conversations**
- Analyze the last conversational turn focusing on **technical** or **quality-of-life** aspects
- Compose **rubrics** defining what an ideal response to your prompt would look like
- Rewrite the model response until it is a **Golden Response**



Your work will **directly improve** the ability of AI to **assist in coding tasks!**

↓ **Let's go!**

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

Table of Contents

1 **Task Walkthrough**
- Task overview
- Step-by-step walkthrough

2 **Rubric Best Practices**
- Principles of rubric criteria
- Weighting

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

01

Task Walkthrough

In this section, we will give an overview of the project’s tasks and a detailed description of each step

Outlier

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

Workflow

Here's how this looks in practice as you task:

1. **Read the conversation**
   - This contains the conversation between user and assistant up to the last prompt
2. **Read the four responses**
   - Pay special attention to the differences between the responses.
3. **Analyze the differences**
   - Take note of the responses' differences, they are not exactly the same!
   - Quality-of-life aspects should be about aspects of an answer that makes it better than the other answers.
4. **Write rubrics**
   - Compose criteria covering code, instruction following, and quality-of-life dimensions that define what an ideal response to the prompt would look like
5. **Grade the responses**
   - Grade whether each response passes each of your rubric criteria
   - Remember that Quality-of-life rubrics must fail in one of the answers.
6. **Score the responses**
   - Based on the rubric grades, assign a 1 – 5 score to each responses. Use the table to guide yourself.
   - Write a brief justification explaining why you assigned the score
7. **Rewrite the final answer**
   - Starting from the better response, rewrite it so that it passes all the criteria

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

Steps

📌 What You'll Do

Step	Task	Details
🔍 **Step I:** Step I: Analyze Conversation & Ratability; Select Prompt Topic & Read 4 Model Responses	Interpret assigned information and assess if the conversation is ratable; Read 4 model responses	• Read the entire conversation and the last user prompt that will be fed to 4 models to generate 4 responses. • Answer the question: "Is the conversation ratable?" based on the definition of ratability • Select a prompt topic based on the prompt and conversation history • Carefully read the 4 model responses, focusing on key differences between them.

📋 **Step II:** Write Rubrics	Create criteria aligned to the failure.	• Read the rubrics created by autogenerator for three rubric categories: Code, Instruction Following (IF), Quality of Life (QoL) to help you write rubrics
		• Write Rubrics & label the rubrics: each rubric has
		  ○ One Criterion (e.g. "Includes a short TL;DR at the end")
		  ○ One Rubric Category (Code or IF or QoL)
		  ○ One Rubric Weight (Mandatory or Nice-to-have)

📊 **Step III:** Grade the Responses	Grade each response using the rubrics you created.	• Rate the 4 model responses one by one, against the rubrics you created in the previous step. • Binary scoring. (Pass or Fail) • Reference observed behavior only.
🏁 **Step IV:** Score the Responses	Assign a 1–5 overall score and write a brief justification.	• Scores must match rubric results and reflect failure logic • Follow the scoring rules • Justification should be comprehensive, factual, and based only on unmet criteria
✨ **Step V:** Select the Best Response & Rewrite the Final Answer	Produce the Golden Response	• Select the best response using overall score and rubrics as reference. • Fix all failed criteria for the selected response, so that your Golden Response fulfills all of them.
📖 **Step VI:** Read the LLM Judges	Read and justify the LLM judges for model Res A and Res B	• We have a LLM judge that analyze if the model response satisfy all the rubrics you created • Read the LLM judge analysis for Res A, Write a 2-3 sentence justification for anything that you disagree with LLM judge for Res A • Read the LLM judge analysis for Res B, Write a 2-3 sentence justification for anything that you disagree with LLM judge for Res B

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

Step 1: Analyze Task Metadata (use case & difficulty)

First, check the assigned use case for your task.

Use Case	Description	Simplified Example
🛠️ **Code Modification & Optimization**	Improving existing code by enhancing performance, readability	"Optimize this Python loop for speed." "Refactor this function to reduce complexity and improve readability."
🐞 **Code Debugging & Troubleshooting**	Identifying, analyzing, and fixing issues, bugs, or unexpected behavior in code.	"Why is this Java method returning null?" "Help me find why this script crashes when the file is empty."
📘 **Code Technical Understanding**	Explaining what code does, how it works, or the reasoning behind a specific implementation.	"Explain this SQL query step by step." "What does this regex pattern mean?"
⚖️ **Code Technical Understanding, Comparisons**	Explaining code while also comparing approaches, patterns, languages, or implementations.	"Compare recursion vs iteration for this function." "Explain the difference between these two implementations and which is better."

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

Step 1: Analyze Conversation & Ratability

Read the conversation, making sure to...

→ Read the entire conversation from the first turn up to the turn immediately before the last one

→ Review all prior turns to understand constraints, context, and prior decisions.

Remember:

→ This conversation is only to acquire the necessary context in order to fully analyze the last turn of the conversation.

→ Tasks with 2 turns will have a conversation history displayed. You should carefully read through it. If it is a 1 turn task, just click "Next" to continue to see the prompt.

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

Step 1: Analyze Conversation & Ratability

✨

Analyze the conversation if it is ratable:

→ Read the conversation and prompt and answer “Is the conversation ratable?”

Prompt Ratability Definition:

❌ **Exclude:**

1. Missing context/turns/files, non-English, unanswerable prompts
2. Non-coding or coding-adjacent questions like system administration, tool installation, OS configuration, and IDE setups.
3. Niche platforms / specific product questions (Unity/Unreal/Roblox/etc; Shopify/WooCommerce/etc; TradingView/MQL; Salesforce/ABAP; Solidity/Web3; etc.)

❌ **Exclude as Too Easy:**

Prompts that are overly simple like “how to split a python string.” Use this option for prompts that can be answered very simply and briefly.

✅ **Include:**

1. Language allowlist (JS/TS/Python/Java/Kotlin/C/C++/C#/Go/Rust/Swift/PHP/SQL/Shell; HTML/CSS optional)
2. Focus on Core Coding: Prioritize questions that are "core" to a programmer's job (e.g., Algorithms, Frameworks, Git, Dockerfile, npm, pip, data analysis, ML engineering, security)

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

Step 2: Read the Prompt & Select a Prompt Topic

Read the user prompt first and select a topic based on the prompt and the entire conversation context. You can **only choose one topic** from the 8 options here:

Topic	Description	Example (high-level)
Code Generation	Generate new functions, application scaffolding, or automation scripts.	"Write a Python script that scrapes all 'h2' tags."
Debugging & Error Fixing	Fix non-working code and resolve specific error messages.	"Why am I getting this TypeError? Here is my code..."
Algorithms & Interview	Standard CS problems, data structures, and interview prep.	"Most optimal solution for 'Two Sum' in JavaScript?"
Config & DevOps	Environment setup, dependency management, and deployment.	"Write me a Dockerfile for a Node.js app."
Code Explanation	Plain-English explanations of existing code, regex, or complex logic.	"Explain this regex to me line by line."
Data Science	Data manipulation and visualization (Pandas, NumPy, R).	"How do I use Pandas to fill missing values?"
SQL & Databases	Database retrieval (queries) and storage design (schemas).	"Write a SQL query to find users who signed up recently."
Refactoring	Improving code efficiency, readability, or maintainability.	"Refactor this nested loop to be more 'Pythonic'."

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

Step 2: Read the Prompt & 4 Model Responses

Read the user prompt first and identify all explicit requirements, including constraints and format expectations.

Evaluate each model response independently against the user prompt and its stated requirements.

User Prompt

- Identify the prompt's concrete requirements, including required actions, constraints, format, and scope.
- Determine the prompt's evaluation goal, clarifying what a correct response is expected to achieve.

Model Responses

- For this turn, **four model responses** will be given: Response A, Response B, Response C and Response D.
  - All responses might have similar quality but not the same. Keep a close eye.
- Analyze all 4 Model Responses and look for Code, Instruction Following, and Quality-of-Life errors to make your rubric later.

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

Step 3: Assess 4 Model Responses

Use these **failure indicators** to assess Model Responses to build your rubrics:

Technical (Code & Instruction Following)

- ❏ Incorrect or missing code logic
- ❏ Misinterpreted instructions affecting implementation
- ❏ Unsafe, misleading, or invalid technical advice
- ❏ Missing required steps or functionality

QoL – Helpfulness

- ❏ Missing explanations or a lack of reasoning
- ❏ Not addressing the user's actual need
- ❏ Ignored user context or follow-up expectations (tutor mode)

QoL – Style & Clarity

- ❏ Hard-to-read explanations
- ❏ Poor code formatting, naming, or structure
- ❏ Disorganized or confusing flow

QoL – Verbosity

- ❏ Overly long, rambling responses
- ❏ Repetitive content
- ❏ Too short to be helpful or missing essential details

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

Step 4: Write rubrics (in total ~10 rubrics)

The goal of your rubric is to write a set of criteria that cover Responses' failures **and** define what a fully correct, ideal response should achieve. The next section will contain best practices for writing rubrics, but for now, note that your criteria will cover the following **three categories:**

Code

→ Evaluates the **correctness, safety,** and **intent** of the code

- Focus on intent, not exact syntax
- Avoid requiring specific methods, libraries, or line-by-line implementations unless explicitly needed
- Accept alternative valid approaches if they meet the goal

Instruction Following (IF)

→ Ensures the model correctly follows the user's **explicit and implicit instructions**

- Check that all parts of the prompt are fully addressed
- No additions, omissions, or scope drift
- No reinterpreting or ignoring user requirements

Instruction Following (IF) rubrics must be mandatory.

Quality of Life (QoL)

→ Evaluates usefulness and clarity across three sub-dimensions:

- **Helpfulness** – relevance, usefulness, guidance, and solving the user's need
- **Style & Clarity** – readable code, clear structure, logical flow, visual organization
- **Verbosity** – concise yet complete; no rambling, no missing essential information

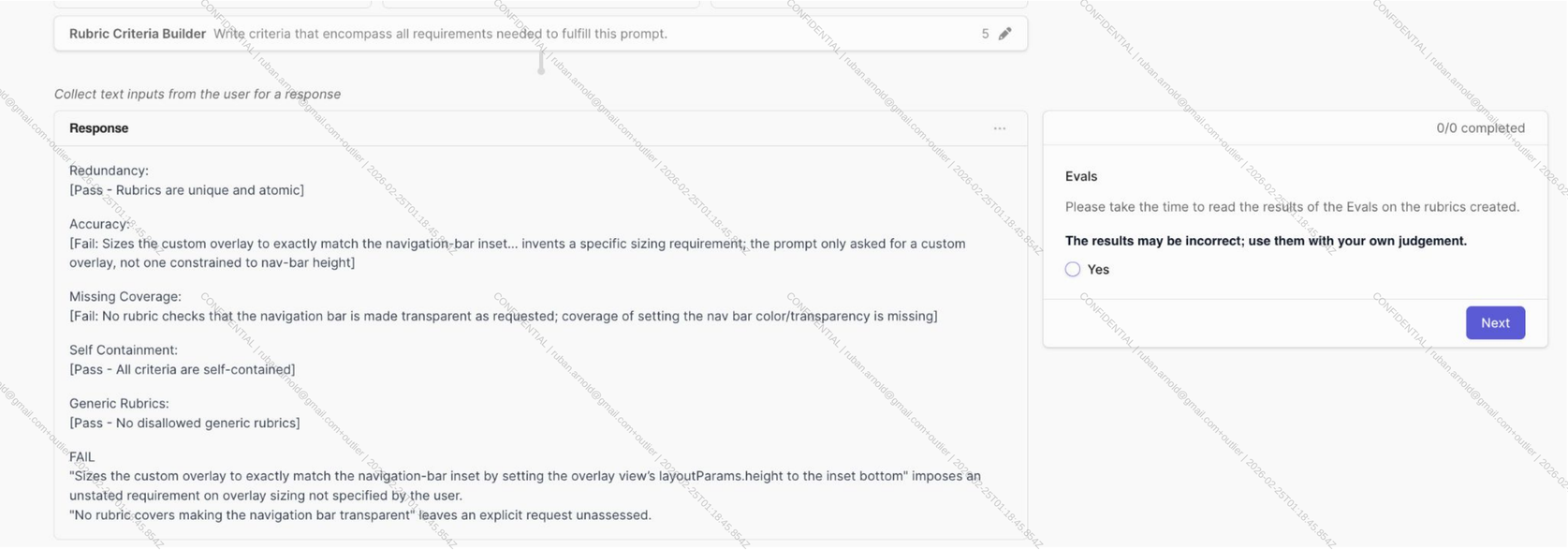
Quality of Life (QoL) rubrics must be Nice-to-have.

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

Working with Evals

Use them as a tool, but always use your own judgement.

Rubric evals are a **helpful tool** when crafting rubric criteria, but the are not an absolute source of truth. Read and consider these suggestions, and make edits to your rubrics based on them when they offer valid feedback.



CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

Step 5: Grade the responses

Grade the 4 model responses on each rubric criterion:

→ Grade each criterion **fulfilled** or **not fulfilled** — no partial credit!

→ Refer **only to the response** and the **rubric** when grading

→ A Quality-of-life criteria **cannot pass** all responses. We are measuring differences!

→ If it is difficult to grade a criterion fulfilled or not fulfilled, you may need to edit the criterion to be clearer and more atomic!

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

Step 6: Score the responses

In this step, you assign an **overall 1 – 5 score** to each model response and provide a short justification **grounded in the rubric scoring.** Make sure to follow the rules below:

5	4	3	2	1
Perfect	**Very good**	**Acceptable**	**Bad**	**Very Bad**
Assign this score **only if** the response **fulfilled every criterion** in your rubric	Assign this score for responses that **fulfilled all code and instruction** following criterion but **failed some QoL** (less than 30%)	The response **failed between 30% and 50%** of **QoL Rubrics.** **OR** The response failed **one mandatory non-QoL Rubric.**	The response **failed more than 50% of QoL Rubrics.** **OR** The response **failed more than one mandatory non-QoL Rubric.**	

1 Code/IF failure = Max Score of 3.

>=2 Code/IF failures = Max Score of 2.

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

Step 7: Select the Best Response & Rewrite the Final Answer

In this step, you will produce the **Golden Response** by **improving the best-scoring model response** selected in previous step. Your rewrite must **correct all issues** identified in the rubric and produce a response that fully meets every criterion.

Start from the best response

→ Choose the response with the highest score on your rubric

→ Do **not** mix responses

→ Do not start from scratch unless both responses failed all criteria

Fix all rubric failures

→ Rewrite the answer so that it **fulfills every rubric criterion**

→ Correct **technical** errors, **follow instructions** fully, and improve the **quality of life** dimensions as needed

Maintain intent & content

→ Keep your Golden Response in the **scope of the original prompt;** no need to take on any tasks that the prompt did not request!

Always remove unnecessary "filler" praise and check the response for contextual accuracy.
"Great use case!" or *"I'd be happy to help!* add no value to a response.
Avoid tone-deaf affirmations: if a prompt contains an error, *"Exactly!"* is not appropriate.

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

Step 8: Read the LLM Judges & write justifications

In this step, an LLM Judge analyzes whether each model response satisfies the specific rubrics you created. You must review the judge's reasoning for **2 model responses (Res A and Res B)** to ensure the evaluation is accurate.

→ Carefully read the Reasoning Trace provided by the LLM Judge for each response.

→ If you find that the LLM Judge has made an error in its decision or justification, you will provide a rebuttal by writing a brief justification. You don’t need to take much time on this step, just explain your logic.

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

02

Rubrics Best Practices

This section will provide some more detail on how to compose robust rubrics in case you are new to rubrics

Outlier

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

A rubric is simply a checklist of criteria attached to a prompt.

To compose a rubric, a data annotator starts with a **prompt.**

The annotator then imagines what an **ideal response** to that prompt would look like.

Finally, the annotator breaks down the **features** of that ideal response into a set of criteria.

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

Principles of Useful Rubrics

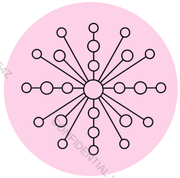
For the model to learn well from a rubric, **it needs to be structured in a very specific way**, or it might learn the wrong things from the rubric.

We can codify the features of a useful rubric as a set of principles.

Rubrics are highly prompt-specific, but...

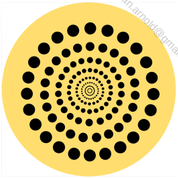
...the goal is a set of criteria that are **mutually exclusive** (they shouldn't overlap) and **collectively exhaustive** (taken together, they should fully define the ideal response).

MECE can be achieved by following **three principles:**



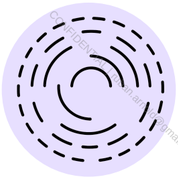
Atomicity

Each criterion should cover **exactly one** aspect



Specificity

Criteria should **precisely** state what is expected



Self-containment

Each criterion should contain **all the information** needed to evaluate a response.

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

Atomicity: Definition and example

Atomicity

Each rubric criterion should evaluate exactly one distinct aspect. Avoid bundling multiple criteria into a single rubric. Most stacked criteria with the word “and” can be broken up into multiple pieces.

What does this look like in a real rubric? ↓

- ✗ The response places code snippets in triple backticks and uses indentation to mark code blocks.
- ✓ The response places code snippets in triple backticks.
- ✓ The response uses indentation to mark code blocks.

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

Specificity: Definition and example

Specificity

Criteria should state precisely what is expected. They should be binary (true or false). Avoid vague descriptions (e.g., "the response must be accurate"): provide more detail to make the criteria as objectively gradable as possible.

What does this look like in a real rubric? ↓

~~"The function outputs the correct answer"~~ → "The function outputs the value 5 when the input is 3."

~~"The variable uses the proper type."~~ → "The variable PRICE is typed as int."

Note how these additional details make the criteria **objectively gradable!**

Make sure, however, that your criteria remain **high level** so that multiple implementations can pass! Do **NOT** prescribe specific implementations **unless they are necessary** to fulfill the prompt.

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

Self-Containment: Definition and examples

Self-Containment

Each criterion should contain **all** the information needed to evaluate a response.
Each criterion should be verifiable **without requiring external research**.

What does this look like in a real rubric?

- ✗ The response uses the library mentioned in the prompt.
- ✓ The response uses the library NumPy.
- ✗ The response uses standard Javascript commenting conventions.
- ✓ The response uses JSDoc comments for API documentation, including using @param to describe function parameters and @returns to describe return values.

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

Criteria weights

Weights

We tell the model the relative importance of different criteria using **weights**.

For this project, you will label each criteria either **mandatory** or **nice to have.**

When should you use these weights?

Mandatory

- Covers **explicit** or **implicit** requirements from the prompt.
- Represents something the response **must include** to be an acceptable response.

Nice to Have

- Improves response quality but is **not required** to fulfill the task.
- Reflects enhancements that **objectively** improve usefulness or clarity.

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

Quality-of-life rubric

What is Quality-of-life?

Quality-of-life rubric (or QoL) measure how *pleasant, usable, and practical* a response is for an user trying to understand or work with the code/information provided.

Although all response might be similar, they **are not the same**. QoL rubric should be used to **analyze, penalize and reward** those differences, assessing which response is better than the others and why.

Because QoL rubric focus on the differences between the responses, these criteria **should not pass** all the responses.

Correctly evaluating QoL

General Helpfulness

- Does the response answer everything the user asked, including implicit requests?
- Are edge cases and realistic usage considered?
- Are key assumptions and next steps clearly stated?

Style & Clarity

- Is the code readable, well-structured, and logically organized?
- Are variable names, comments, and formatting clear and consistent?
- Is supporting documentation (docstrings, I/O notes, complexity) included if needed?

Verbosity

- Does the response avoid repetition and irrelevant filler?
- Is the technical level appropriate for the user?

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

Finding the Balance: Prescriptive vs. Generic

🛑 1. Avoid: Overly Prescriptive (Rigid) Rubrics

The Test: If a model provides a different but technically sound solution (different library/pattern), would this rubric force a failure?

- **Why it's bad:** It penalizes "correct" models for not being mind-readers.
- **Exceptions:** Only be prescriptive if the prompt explicitly demands a method (e.g., "Use a `for` loop") or for security (e.g., "Use parameterized queries").

☁️ 2. Avoid: Overly Vague (Generic) Rubrics

The Test: If you swapped this Python prompt for a Java prompt, would the rubric still work?

- **Why it's bad:** "Code is clean" or "Provides a summary" are filler. They don't evaluate the model's coding logic.

💡 The "Technical Teeth" Strategy

- **Use "LIKE / e.g.":** Give the judge a standard while remaining flexible.
  - *Bad:* "Uses initializer injection."
  - *Good:* "Uses a standard form of Dependency Injection (e.g., Initializer, Property, or Method injection)."
- **Provide Ground Truth:** Since the LLM Judge doesn't "know" the answer, you must define it.
  - *Bad:* "Decodes the file correctly."
  - *Good:* "Identifies the compression algorithm (LZ-String) and method (decompressFromBase64) to deserialize the format."

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

Update

a. **Golden Response & Tone Standards:**
   i. **Remove Unnecessary Praise**: Golden Responses must be professional and direct. Delete "filler" praise such as "Great question!", "Great use case!", or "I'd be happy to help!".
   ii. **Contextual Accuracy**: Avoid tone-deaf affirmations. For example, if a user shows an error, the response should not say "Exactly!". CBs should remove those in the final golden response!

b. **Rubric Scaling & Complexity**
   i. Simple Prompt: For low-complexity prompts (e.g., "what is 1<<n in cpp"), do not need too many rubrics. E.g we have seen a 13 rubrics set for a one sentence easy prompt, this is not needed.

c. **Negative rubrics**
   i. Avoid negative rubrics like “the response Does NOT have …”

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z

Congrats, you made it!

You’re ready to judge and improve the coding abilities of AI!

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-02-25T01:18:45.854Z